Data Consistency in MongoDB Lab

Introduction

This lab demonstrates how MongoDB ensures data consistency using transactions and write concerns. It highlights how failures can lead to inconsistent data and how transactions solve this problem by ensuring atomic operations.

Step 1: Transfer Without Transactions

This step shows a basic transfer between accounts without using transactions, which may lead to inconsistent data if a failure occurs.

```
db.accounts.updateOne({ _id: 1 }, { $inc: { balance: -20 } })
db.accounts.updateOne({ _id: 2 }, { $inc: { balance: 20 } })
```

Step 2: Failure Scenario

A simulated error interrupts the transfer, causing inconsistency because only one operation is executed.

```
db.accounts.updateOne({ _id: 1 }, { $inc: { balance: -20 } })
throw new Error("Simulated failure")
db.accounts.updateOne({ _id: 2 }, { $inc: { balance: 20 } })
```

Step 3: Transactions

Transactions ensure both operations succeed or fail together, maintaining consistency.

```
const session = db.getMongo().startSession() session.startTransaction()
try {   const accounts =
session.getDatabase("testdb").collection("accounts")

  accounts.updateOne({ _id: 1 }, { $inc: { balance: -20 } }, { session })
accounts.updateOne({ _id: 2 }, { $inc: { balance: 20 } }, { session })

  session.commitTransaction()
} catch (error) {
  session.abortTransaction()
} finally {
  session.endSession()
}
```

Step 4: Transaction with Failure

If an error occurs inside a transaction, all changes are rolled back automatically.

```
throw new Error("Simulated failure")
```

Step 5: Write Concerns

Write concerns control how many nodes confirm a write. Higher levels improve durability but reduce performance.

```
db.inventory.insertOne({ item: "notebook" }, { writeConcern: { w: 1 } })
db.inventory.insertOne({ item: "pencil" }, { writeConcern: { w: "majority" } })
```

Observations

Without transactions, data becomes inconsistent during failures. Transactions ensure atomicity. Write concerns affect reliability and speed.

Answers

Transactions ensure all operations succeed or fail together, preventing partial updates. Write concern trade-offs: higher durability vs slower performance. Consistency priority: banking systems. Performance priority: logging systems.

Conclusion

This lab shows that transactions and write concerns are essential for maintaining data integrity in MongoDB. Choosing the right balance between performance and consistency is crucial.